AARHUS UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

COMPUTER ENGINEERING

Term:	FALL 2021, reexamination
Examination:	Introduction to Programming for Computer Engineers
Date:	3 June 2022
Duration:	3 hours, 9:00 – 12:00
Room:	room number(s)

1 cover plus paper for draft notes will be handed out.

REMEMBER to state name and study number on all pages of your submission.

Remember to show us an ID when returning the present exam form. We will then mark your name on our master list.

- This document is split into 5 sections focusing on different topics of the lecture. Each section contains up to 5 questions. You have 180 minutes to answer all of them. In addition, there is one challenging bonus question that you can work on if you get the time, since it will be possible to get the highest mark even without answering the bonus question, but it can compensate for mistakes you have done in the first 5 sections. Strategically, you should first scan it quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are multiple choice questions, questions that require you to write C code and drawings. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt what is expected, please ask!**
- The multiple choice questions may have any combination of correct answers. All choices may be correct, none or anything in between.
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.
- At the end of the exam paper (Section 6) you find one bonus question worth 10 points. You are ***not*** required to solve it and can attain the full score of 100 points ***without*** it. It is harder to solve but it can be used to compensate for mistakes in answers to other questions.

1 Types and Syntax

Question 1.1 **[5]**

What type do the following values belong to? Indicate your answer by ticking the boxes on the right.

	`int`	`int[2]`	`char`	`char[2]`	other
• `2.2`	☐	☐	☐	☐	☑
• `'2'`	☐	☐	☑	☐	☐
• `30000`	☑	☐	☐	☐	☐
• `{ -4, 3 }`	☐	☑	☐	☐	☐
• `{ "4","3" }`	☐	☐	☐	☐	☑
• `{ '4','3' }`	☐	☐	☐	☑	☐

Question 1.2 **[5]**

Which of the following lines are syntactically correct (tick the correct box(es), `a` is a variable of type `int`)?

☑ `if (a==0) a=1;`

☐ `if (a=0) a==1;`

☐ `if (a==0) a=1.0;`

☐ `if (a<>0) a=1;`

☐ `if (a==0) a=1`

Question 1.3 [5]

A function `volume` calculates and returns the volume of a cylinder as a float. The input is given as two floats: `diameter`, `height`. You must use the function `area` in your implementation of `volume`. Complete the function `volume` below.

```
#define PI 3.1415
float area (float diameter) {
  return diameter*PI;
}

float volume (float diameter, float height) {

  return area(diameter)* height;
}
```

Question 1.4 [4]

The function `secret` takes a pointer to an integer and adds 7 to the integer value which it points to. The function does not return any value. Complete the function signature and function body below, so that the main function prints 19 when executed.

```
void secret (int*p) {

  *p = *p + 7;
}

int main () {
  int v = 12;
  secret(&v);
  printf("%d", v);
  return 0;
}
```

2 Basic Programs

Question 2.1 [3]

Let q, w, t be integer variables declared by

```
int q;
int w;
int t;
```

Which assignment (a), (b), (c) or (d) inserted for the question mark in the program fragment below makes the following program fragment print out 10:

```
q = 2;
w = 4;
?
t = q + t;
printf("%d", t)
```

Answer:

(a) `w = q * w;` ☐
(b) `t = t * w;` ☐
(c) `t = 8;` ☑
(d) `t = q * w;` ☑

Tick the box on the right of the assignment to indicate by which assignment to replace the question mark in the program fragment. More than one expression may be correct.

Question 2.2 [3]

Consider the following program fragment

```
int i;
int b[] = {0,1,2,3,4,5,6,7};
for(i = 0; i < 6; i = i + 1)
  b[i] = b[i] + b[i+2];
```

Fill in the table below with the values stored in array b after the program has been executed.

Answer: b =

2	4	6	8	10	12	6	7

Question 2.3 **[4]**

Consider this program fragment

```
int c = a+b;
int d = 1;
if(b == 1)
  c = 1;
if(a == 1 || b == 0)
  d = 0;
d = c * d;
printf("%d", d);
```

For the values of variables a and b as indicated below what is the resulting value of variable d (i.e. what is printed out)?

a	b	d				
1	1	-1 ☐	0 ☑	1 ☐	2 ☐	someting else ☐
1	0	-1 ☐	0 ☑	1 ☐	2 ☐	someting else ☐
0	1	-1 ☐	0 ☐	1 ☑	2 ☐	someting else ☐
0	0	-1 ☐	0 ☑	1 ☐	2 ☐	someting else ☐

Tick the box next to the correct value for d with respect to the initial values of a and b as stated in the first two columns of the table.

Question 2.4 **[4]**

Consider the following program:

```
1  #include <stdio.h>
2
3  int fac(int i) {
4
5    return fac(i-1)*i;
6
7  }
8
9  int main(void) {
10
11   printf("%d", fac(3));
12
13   return 0;
14
15 }
```

it can compile but it will not produce the expected result (6 = 3*2*1)

What is the problem and how will you change to program so it will correctly calculate the factorial (include the line number(s) where you will make the change(s))?

The error is: no base case so fac will not terminate

It is corrected by: 4: if (n==1) return 1 else

Question 2.5 **[6]**

Consider the program fragment below.

```
int x;
int y;
int z;

x = ?
y = ?
z = ?

switch(x) {
  case 1:
    x = y*z;
    z = z*2;
    break;
  case 2:
    y = x + z;
    break;
  default:
    {
      int z;
      z = x + y;
      x = y * z;
    }
    break;
}

printf("[%d, %d, %d]", x, y, z);
```

For each of the following assignments for variables x, y, z, write the three integer numbers that will be printed when the program is executed.

(1) x=0, y=1, z=2:

Answer: [1 , 1 , 2]

(2) x=1, y=2, z=3:

Answer: [6 , 2 , 6]

3 While Programs

Question 3.1 **[6]**

You are given the declarations of variables `w` and `i`, and the initialisation of them.

```
int i;
int w;
w = -1;
i = 2;
```

You are given the following while-loop:

```
while(w <= 6) {
  w = w + 2;
  i = i + 1;
}
```

Fill in the blanks below so that the resulting for-loop computes the same values for `w` and `i` as the while-loop above. You are only allowed to use variable declarations, assignments and conditions that already occur in the while-loop (some lines below can be empty).

```
_____
int i;
i = 2;
for (int w = -1; w <= 6; w = w+2) {
  i = i + 1;
}
```

Question 3.2 **[6]**

What do the following program fragments print out at the end of their execution where `i` and `x` are integer variables?

Program fragment 1:

```
int x = 2;
for (int i=0; i > 3; i = i + 1)
  x = i*2;
printf("%d", x);
```

Program fragment 2:

```
int x = 2;
for (int i=3; i >= 3; i = i+1)
  x = i*2;
printf("%d", x);
```

Program fragment 3:

```
int x = -2;
for (int i=10; i > 7; i = i - 1)
  x = x + 1;
printf("%d", x);
```

Indicate your answer by ticking the column with the correct result in the table below. The column 'no result' signifies that no value is printed at all (indefinite loop).

	1	2	3	4	5	6	7	8	9	no result
Program fragment 1		X								
Program fragment 2										X
Program fragment 3	X									

Question 3.3 **[6]**

Consider the following program fragment.

```
int x;
int y;
int p;
int r;
scanf("%d", &y);
scanf("%d", &x);
if (y<1)
  abort();
p=1;
r=x;


_____________________ /* Precondition goes here */

while (p<y){

  _____________________ /* Loop Invariant goes here */

  r=r*x;
  p=p+1;
}

_____________________ /* Postcondition goes here */
```

Tick the following boxes such that the property specified for the column holds (as an assertion) at the program location specified by the row. Tick all true statements for each row (from 0 to 6 ticks).

	y>=1	1>=y	p=y	r=x	$r=x^p$	$r=x^y$
Precondition	☒	☐	☐	☒	☒	☐
Loop Invariant	☒	☐	☐	☐	☒	☐
Postcondition	☒	☐	☒	☐	☒	☒

Question 3.4 **[8]**

For each program below, let N be any positive integer. After executing each program, what is the value of v as a function of N?

Program 1:

```
int v = 0;
for(int i = 1; i <= N; i = i + 1) {
  v = v;
}
```

Program 2:

```
int v = 0;
for(int j = 1; j <= N; j = j + 1) {
  for(int i = 1; i < 1; i = i + 1) {
    v = v + 1;
  }
}
```

Program 3:

```
int v = 0;
for(int i = 1; i <= N; i = i + 1) {
    v = v + 1;
}
```

Program 4:

```
int i;
int v = 0;
for(i = 1; i <= N; i = i + 1) {
  v = v + i;
}
```

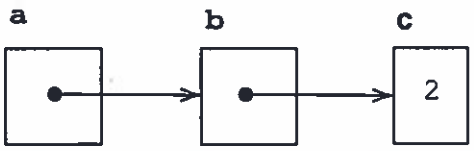
Tick the boxes below to indicate the value of v as a function of N (column) for each program (row). *Hint*: exactly one answer is correct per program.

	v=0	v=N	v=2*N	$v=N^2$	$v=\frac{N(N-1)}{2}$	2^N
Program 1	☒	☐	☐	☐	☐	☐
Program 2	☒	☐	☐	☐	☐	☐
Program 3	☐	☒	☐	☐	☐	☐
Program 4	☐	☐	☐	☐	☒	☐

4 Pointer Programs

Question 4.1 **[4]**

The following pictures (in the notation from the lecture) shows two variables (a, b and c) and their values.



Fill in the rest of the five lines so that the declaration and assignment results in the illustration:

```
int*a____;

int *b____;

 int c____;
int main(void) {

  c= 2_____;

  b=&c_____;

  a=&b_____;
}
```

Hint: Some part(s) of the lines might be blank

Question 4.2 **[4]**

Consider the following program fragment:

```
  int x;
  int y;
  int z;
  int* w;
  int* q;
  x = 0;
  y = 1;
  z = 2;
  w = &z;
  q = w;
  z = 3;
  *q = y;
  *w = z;
  printf("x=%d, y=%d, z=%d",x,y,z);
```

What does the program print when it is executed?

Answer: x = 0, y = 1, z = 1

Question 4.3 **[7]**

Consider the following program fragment:

```
1  typedef struct node {
2     int data;
3     struct node* go_on;
4  } node;
5
6
7  node* new_node(int v, node* l)
8  {
9   node *n =(node*)malloc(sizeof(node));
10  n->data = v;
11  n->go_on = l;
12  return n;
13 }
14
15 node* rt(node* n) {
16  if(n == NULL) {
17     return NULL;
18  }
19  else {
20     return new_node(n->data*2, n);
21  }
22 }
23
24 void print(node* n){
25   if (n!=NULL) {
26     printf("%d, ",n->data);
27     print(n->go_on);
28   } else
29     printf("\n");
30 }
31
32 int main () {
33   node* a = new_node(2, NULL);
34   a = rt(a);
35   print(a);
36   a = a->go_on->go_on;
37   a = rt(a);
38   print(a);
39 }
```

What is the output of the program?

line 35: 4, 2,

line 38: nothing \ blank line \ new line \ "\n"

in general, missing the newline should not deduct points

5 Recursive Programs

Question 5.1 **[4]**

You are given the following program fragment

```
int recur(int n){
  assert(n>=0);
  int t = n;
  if (n==0)
    return t;
  else
    return t+ recur(n-2);
}

int main(void) {
  printf("recur(4)=%d", recur(4));
}
```

What does it print out? Tick the corresponding box below.

- ☐ recur(4)=0
- ☐ recur(4)=1
- ☐ recur(4)=2
- ☐ recur(4)=4
- ☒ recur(4)=6
- ☐ recur(4)=10
- ☐ something other than the above

Question 5.2 **[4]**

What is the missing assertion and base case of the following function for computing the sum from 10 to `n` (that is $\sum_{i=10}^{n} i$)?

```
int sum(int n) {
    ______________
    ______________
    else {
        return n + sum(n - 1);
    }
}
```

Tick the appropriate program fragment to complete the if-statement in function `sum`.

- ☐ `assert(n == 0);`
  `if(n == 0) return 0;`
- ☐ `assert(n >= 0);`
  `if(n == 0) return 0;`
- ☐ `assert(n == 10);`
  `if(n == 1) return 1;`
- ☒ `assert(n >= 10);`
  `if(n == 10) return 10;`
- ☐ `assert(n > 10);`
  `if(n == 10) return 10;`
- ☐ `assert(n >= 10);`
  `if(n == 1) return 1;`

Question 5.3 **[12]**

You are given the following declaration of a tree datatype, where the empty tree corresponds to the pointer value `NULL`, the field `data` stores the value at a node of the tree, and `left`, `right`, are the left and right children.

```
typedef struct node
{
  int data;
  struct node* left;
  struct node* right;
} node;
```

Complete the missing code in the recursive `minimum` function so that it returns the minimum value (`data`) in the tree `t`.

```
int minimum(node *t) {

  assert( t != NULL );

  if( t->left == NULL && t->right == NULL)

    return t->data ;

  else {
      int left_min = INT_MAX;
      int right_min = INT_MAX;
      if (t->left != NULL)
        left_min = minimum(t->left);
      if(t->right != NULL)
        right_min = minimum(t->right);
    return min(left_min, right_min) ; }
}
```

You can write more than one line in the `else` part. If you need to initialize variables to the smallest/largest integer, you can use `INT_MIN` and `INT_MAX`.Furthermore, you can use the following function to find the minimum of two integers:

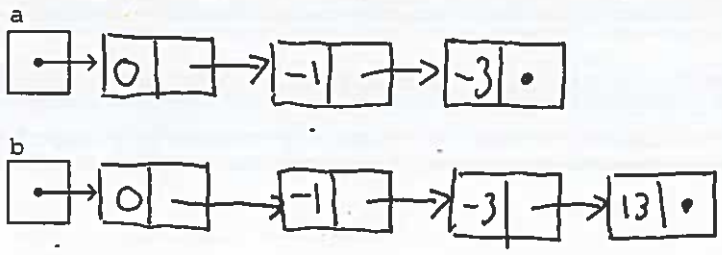
```
int min(int x, int y){
  if (x<y)
    return x;
  else
    return y;
}
```

6 Bonus

Question 6.1 [10]

NEW: Consider the following program fragment:

```c
typedef struct node {
   int content;
   struct node *next_node;
} node;


void change_it(node* a, node* b) {
  if(a != NULL && b != NULL) {

      if(a->content >0)
          a->content = b->content;
      else if(b->content >0)
          b->content = a->content;

      change_it(a->next_node, b->next_node);
    }
}


node* new_node(int v, node* l) {
  node *n = (node*) malloc(sizeof(node));
  n->content = v;
  n->next_node = l;
  return n;
}
```

```c
int main () {
  node* a = new_node(0,
              new_node(1,
                new_node(-3,NULL)
            ));

  node* b = new_node(0,
              new_node(-1,
                new_node(11,
                  new_node(13,NULL)
            )));


  change_it(a,b);

// draw a and b here

}
```

Draw, in the same way as drawn in the lectures and question 4.1, the values of the allocated structs (a and b) after `main` has been executed (remember to include `NULL`):